Documentação da Arquitetura do Projeto Laravel

Estrutura da Arquitetura

```
App/
├── Http/
│   ├── Controllers/
│   │   ├── AbstractController.php    # Camada base para Controllers
│   ├── Middleware/                   # Middlewares da aplicação
│   ├── Requests/                     # Form Requests para validação
│
├── Services/
│   ├── AbstractService.php           # Camada base para Services
│
├── Repositories/
│   ├── AbstractRepository.php        # Camada base para Repositories
│
├── Traits/                           # Traits reutilizáveis
```

```
routes
├── dir_name/
│   ├── file_rota.php                 # Arquivo de rotas de um modulo especifico
```

Essa estrutura segue um padrão de separação de responsabilidades, buscando um acoplamento fraco e maior reaproveitamento de código.

Descrição das Camadas

Controllers

Responsáveis por orquestrar a requisição e direcioná-la à camada de serviço. Os controllers herdam de AbstractController, centralizando comportamentos comuns.

Services

Camada onde reside a lógica de negócio. Cada Service é responsável por uma entidade e herda comportamentos genéricos de AbstractService.

Repositories

Responsáveis pelo acesso aos dados. Cada Repository interage diretamente com um Model e herda de AbstractRepository, que encapsula métodos comuns como find, create, update, etc.

Requests

FormRequests responsáveis por validações de entrada. Cada entidade possui um Request específico com regras definidas.

Traits

Local para armazenar funcionalidades reutilizáveis entre diferentes classes.

Princípios Aplicados

Separation of Concerns (SoC)

- Cada camada tem uma responsabilidade clara:
- Controller → Entrada da requisição
- Service → Regras de negócio
- Repository → Acesso aos dados
- Request → Validação

Princípios SOLID

- **S — Single Responsibility Principle (SRP)**
  Cada classe tem uma única responsabilidade: Controller, Service, Repository, etc.

- **O — Open/Closed Principle (OCP)**
  Com uso de herança (Abstract*) e Traits, o sistema pode ser estendido sem modificar diretamente as classes existentes.

- **L — Liskov Substitution Principle (LSP)**
  Os Abstract* garantem que qualquer classe filha possa ser usada em seu lugar sem quebrar o sistema.

- **I — Interface Segregation Principle (ISP)**
  Cada camada depende apenas do que ela precisa — evitando classes gigantes com métodos que não são usados.

- **D — Dependency Inversion Principle (DIP)**
  Controllers recebem serviços por injeção de dependência, e serviços recebem repositórios da mesma forma, promovendo baixo acoplamento.

Regras de Nomenclatura

Tipo	**Formato**	**Exemplo**
Controller	NomeController.php	UserController.php
Service	NomeService.php	UserService.php
Repository	NomeRepository.php	UserRepository.php
Request	NomeRequest.php	UserRequest.php
Trait	NomeTrait.php	HasTimestampsTrait.php
Abstracts	Abstract*.php	AbstractService.php
Diretórios	Singular e PascalCase	App/Service/User/